C++

程式語言（二）實習課

Introduction to Programming (II) Practice Lessons

Structures

Joseph Chuang-Chieh Lin

Dept. CSE, NTOU

Platform/IDE

- Dev-C++

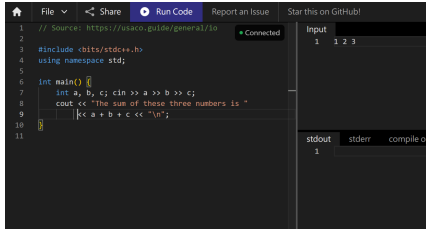
https://www.pngegg.com/en/search?q=Dev-C

- Codeblocks

https://icons8.com/icons/set/code-blocks

- OnlineGDB (https://www.onlinegdb.com/)

- Real-Time Collaborative Online IDE (https://ide.usaco.guide/)



Useful Resources

- Tutorialspoint
  - https://www.tutorialspoint.com/cplusplus/index.htm
  - Online C++ Compiler
- Programiz
  - https://www.programiz.com/cpp-programming
- LEARN C++
  - https://www.learncpp.com/
- MIT OpenCourseWare - Introduction to C++
  - https://ocw.mit.edu/courses/6-096-introduction-to-c-january-iap-2011/pages/lecture-notes/
- Learning C++ Programming
  - https://www.programiz.com/cpp-programming
- GeeksforGeeks
  - https://www.geeksforgeeks.org/c-plus-plus/

Course TAs

- 林琮祐
  - Email address: 11357051@o365.ntou.edu.tw
- 張意苹
  - Email address: 11457070@o365.ntou.edu.tw
- 楊昀潼
  - Email address: 11457074@o365.ntou.edu.tw

Structure

Define a customized data type: **struct**

```
struct Person {
    char name[50];
    int age;
    float salary;
};
```

Three members of Person:

- name
- age
- Salary

No memory is allocated so far.
Just a blueprint for creating variables of such a datatype.

Define a customized data type: **struct**

```
struct Person {
    char name[50];
    //string name;
    int age;
    float salary;
};
```

```
Person bill;
bill.age = 50;
bill.salary = 10000;

cout << bill.age;
cout << bill.salary;
```

Three members of Person:

- name
- age
- Salary

No memory is allocated so far.
Just a **blueprint** for creating variables of such a datatype.

Another Example: `Student`

```
struct Student {
    string name;
    int id;
    string phone;
    // why use string?
    float GPA;
    int birthYear;
    int birthMonth;
    int birthDay;
};
```

```
Student josef;
```

```
josef.name = "Josef Kunze";
josef.id = 12345;
josef.phone = "+49-01245678";
josef.GPA = 3.5;
josef.birthYear = 1965;
josef.birthMonth = 9;
josef.birthDay = 30;

cout << "Student: " << josef.name << endl;
cout << "ID: " << josef.id << endl;
cout << "Phone: " << josef.phone << endl;
cout << "GPA: " << josef.GPA << endl;
cout << "Birthday: " << josef.birthYear << "/"
                    << josef.birthMonth << "/"
                    << josef.birthDay
                    << endl;
```

List initialization

```
struct Student {
    string name;
    int id;
    string phone;
    // why use string?
    float GPA;
    int birthYear;
    int birthMonth;
    int birthDay;
};
```

```
Student josef { "Josef Kunze",
                12345,
                "+49-01245678",
                3.5,
                1965,
                9,
                30 };

cout << "Student: " << josef.name << endl;
cout << "ID: " << josef.id << endl;
cout << "Phone: " << josef.phone << endl;
cout << "GPA: " << josef.GPA << endl;
cout << "Birthday: " << josef.birthYear << "/"
                     << josef.birthMonth << "/"
                     << josef.birthDay << "/"
                     << endl;
```

Using pointers to assess object values

```
struct Student {
    string name;
    int id;
    string phone;
    // why use string?
    float GPA;
    int birthYear;
    int birthMonth;
    int birthDay;
};
```

```
Student josef { "Josef Kunze",
                12345,
                "+49-01245678",
                3.5,
                1965,
                9,
                30 };
auto ptr { &josef };
cout << "Student: " << ptr->name << endl;
cout << "ID: " << ptr->id << endl;
cout << "Phone: " << ptr->phone << endl;
cout << "GPA: " << ptr->GPA << endl;
cout << "Birthday: " << ptr->birthYear << "/"
                    << ptr->birthMonth << "/"
                    << ptr->birthDay << "/"
                    << endl;
```

Passing a structure to a function

```
struct Person {
    string name;
    int age;
    float salary;
};

void displayData(Person);

int main() {
    Person p;

    cout << "Enter Full name: ";
    cin.get(p.name, 50);
    cout << "Enter age: ";
    cin >> p.age;
    cout << "Enter salary: ";
    cin >> p.salary;

    // Function call with structure variable as an argument
    displayData(p);

    return 0;
}
```

```
void displayData(Person p) {
    cout << "\nDisplaying Information." << endl;
    cout << "Name: " << p.name << endl;
    cout << "Age: " << p.age << endl;
    cout << "Salary: " << p.salary;
}
```

Passing a structure to a function

```
struct Person {
    string name;
    int age;
    float salary;
};

void displayData(Person);

int main() {
    Person p;

    cout << "Enter Full name: ";
    cin.get(p.name, 50);
    cout << "Enter age: ";
    cin >> p.age;
    cout << "Enter salary: ";
    cin >> p.salary;

    // Function call with structure variable as an argument
    displayData(p);

    return 0;
}
```

```
void displayData(Person p) {
    cout << "\nDisplaying Information." << endl;
    cout << "Name: " << p.name << endl;
    cout << "Age: " << p.age << endl;
    cout << "Salary: " << p.salary;
}
```

Question:
Any other approach?

Member function of a structure

```
struct Person {
    string name;
    int age;
    float salary;
    void displayData();
};


int main() {
    Person p;

    cout << "Enter Full name: ";
    getline(cin, p.name);
    cout << "Enter age: ";
    cin >> p.age;
    cout << "Enter salary: ";
    cin >> p.salary;

    // Function call with structure variable as an argument
    p.displayData();

    return 0;
}
```

```
void Person::displayData() {
    cout << "\nDisplaying Information." << endl;
    cout << "Name: " << name << endl;
    cout << "Age: " << age << endl;
    cout << "Salary: " << salary;
}
```

https://onlinegdb.com/PogwpWS0A

Exercise 01 – Rational Numbers (1/2)

- Given `rational.h`: and `rational-header.cpp`: (adding some headers...)

```
struct Rational {
    int q, p;
};
```

```
int gcd (int a, int b) {
    return (a % b == 0) ? b : gcd(b, a% b);
}
void simplify(Rational &a) {
    /* complete this function e.g., 4/6 -> 2/3 */
}
Rational ADD(const Rational &a, const Rational &b) {
    /* complete this function */
}
Rational SUB(const Rational &a, const Rational &b) {
    /* complete this function */
}
Rational MUL(const Rational &a, const Rational &b) {
    /* complete this function */
}
Rational DIV(const Rational &a, const Rational &b) {
    /* complete this function */
}
void print(Rational &a) { /* complete this function */}
```

Exercise 01 – Rational Numbers (2/2)

- Driving main function:

```
int main() {
    Rational a, b, c;
    cin >> a.q >> a.p >> b.q >> b.p;
    c = ADD(a, b);
    print(c);
    cout << endl;
    c = SUB(a, b);
    print(c);
    cout << endl;
    c = MUL(a, b);
    print(c);
    cout << endl;
    c = DIV(a, b);
    print(c);
    return 0;
}
```

Sample input:

```
4 7 2 3
```

Sample output:

```
26/21
-2/21
8/21
6/7
```

Exercise 02 – Student (1/3)

- Use the following structures and following the requirements to write a C++ program.

```
struct Date {
    int year;
    int month;
    int day;
};
```

```
struct Student {
    string name;
    int id;
    float GPA;
    Date birthDate;
};
```

```
int main() {
    Student s;
    /* prompt the user to input the
       information of student s
    */
    print(s);
    /* print the information out */
    return 0;
}
```

Exercise 02 – Student (2/3)

Sample input:

```
Elsa
12345
4.5
19890328
```

⇨

Sample output:

```
Name: Elsa
ID:12345
GPA: 4.5
BIRTHDAY: 1989-03-28
```

Sample input:

```
John von Neumann
123456
4.9
19031228
```

⇨

Sample output:

```
Name: John von Neumann
ID:123456
GPA: 4.9
BIRTHDAY: 1903-12-28
```